HW4 flashAttention

114062640 謝秉宸

1. Implementation

a. Describe how you implemented the FlashAttention forward pass using CUDA.

利用 CUDA 的 Memory Hierarchy 特性來減少對 High Bandwidth Memory (HBM) 的存取次數。
1. Tiling (分塊)：我將輸入矩陣 Q, K, V切分為較小的 Blocks。原先為N*d，現在切為每小塊 B_r *d
Q的一個 Block (B_r *d) 被載入到 Registers (暫存器) 中，因為每個 Thread 負責計算 Q的一個 Row。
K和 V的 Blocks (B_c *d) 被載入到 Shared Memory (SRAM) 中。這允許 Block 內的所有 Threads 重複存取 K和 V數據，而無需反覆讀取 Global Memory 1。
2. Kernel Fusion：矩陣乘法 (QK^T)、Softmax 的指數計算、縮放 (Scaling) 以及與V的乘積運算都在同一個 Kernel 內完成，避免將巨大的 N*N Attention Matrix 寫回 HBM 2。

b. Explain how matrices Q, K, and V are divided into blocks and processed in parallel.

Q、K、V皆是由上往下切
將 Q 矩陣沿著 N 維度切分為 T_r 個區塊 (Blocks) 1，每個區塊的大小為 $B_r \times d$。
$T_r = \lceil \frac{N}{B_r} \rceil$，其中 B_r 是Block Row Size (64)。
將 K 和 V 矩陣沿著 N 維度切分為 T_c 個區塊 。每個區塊的大小為 $B_c \times d$。
$T_c = \lceil \frac{N}{B_c} \rceil$，其中$B_c$ 是Block Column Size (64)。
這些 K/V 區塊是用於 **Tiling**。在 Kernel 內部，程式會依序載入這些block到 SRAM中與 Q 進行計算。

c. Describe how you chose the block sizes B_r and B_c and why.

```
#define BR 64
#define BC 64
```

我選擇了 $B_r = 64$ 和 $B_c = 64$。最初嘗試 $B_r = 32$，但似乎導致 GPU Occupancy 過低。提升到 64 (2個 Warps) 配合 `__launch_bounds__(64)` 可以讓 GPU 排程器更有效地隱藏記憶體延遲，並且在速度上也小幅加速。較大的 B_c 代表從 Global Memory 讀取一次 Tile 後，可以進行更多次的乘加運算 (64×64 次)，提高了計算密度。

d. Specify the configurations for CUDA kernel launches

Threads per Block: 64 (對應 B_r)。

Grid Dimensions: `dim3(B, (N + Br - 1) / Br)`。
x軸處理 Batch Size。
y軸處理 Query 的分塊數量 (T_r)。

Shared Memory Allocation:
宣告：`__shared__ float s_K[64][64 + 4]` 和 `__shared__ float s_V[64][64 + 4]`。並使用 **Padding (+4)** 來解決 Bank Conflict。

e. Justify your choices and how they relate to the blocking factors and the SRAM size.

首先查看GPU SRAM

```
=== GPU Device Properties ===
Device Name: NVIDIA GeForce GTX 1080
Compute Capability: 6.1
Max Shared Memory Per Block: 49152 bytes (48.00 KB)
Max Threads Per Block: 1024
```

接著計算 $2 \times 64 \times (64 + 4) \times 4 \text{ bytes} \approx 34.8 \text{ KB}$。34.8KB小於48KB，所以這個配置是安全的。

2. Profiling Results

```
==2053852== Metric result:
Invocations                                Metric Name                        Metric Description         Min         Max         Avg
Device "NVIDIA GeForce GTX 1080 (0)"
    Kernel: flash_attention_kernel_final(float const *, float const *, float const *, float*, int, int, float)
        100                        achieved_occupancy                        Achieved Occupancy    0.057266    0.062493    0.059395
        100                                sm_efficiency                   Multiprocessor Activity      80.07%      87.40%      84.18%
        100                     shared_load_throughput             Shared Memory Load Throughput  791.72GB/s  795.52GB/s  793.87GB/s
        100                    shared_store_throughput            Shared Memory Store Throughput  16.494GB/s  16.573GB/s  16.539GB/s
        100                                gld_throughput                    Global Load Throughput  33.246GB/s  33.406GB/s  33.337GB/s
        100                                gst_throughput                   Global Store Throughput  263.91MB/s  265.17MB/s  264.62MB/s
```

由圖中可見雖然achieved occupancy較低，但sm efficiency相當高，應該是pipeline的設計掩蓋了memory的延遲，才使使用率那麼高，此外，可以注意到Shared Memory 的讀取頻寬 (794 GB/s) 是 Global Memory (33 GB/s) 的 **24 倍**。代表將矩陣做tiling有其必要性。

```
==2061655== Profiling application: ./hw4 /home/pp25/share/hw4/testcases/t24 out.bin
==2061655== Profiling result:
            Type  Time(%)      Time     Calls       Avg       Min       Max  Name
 GPU activities:   96.38%  982.12ms       100  9.8212ms  7.8355ms  11.459ms  flash_attention_kernel_final(float const *, float const *, float const *, float*,
)
                    2.81%  28.653ms       300  95.509us  90.144us  109.12us  [CUDA memcpy HtoD]
                    0.81%  8.2113ms       100  82.113us  80.609us  83.776us  [CUDA memcpy DtoH]
      API calls:   65.36%  623.01ms         1  623.01ms  623.01ms  623.01ms  cudaDeviceSynchronize
                   27.83%  265.25ms         4  66.313ms  44.639ms  126.84ms  cudaMallocHost
                    5.52%  52.586ms         4  13.146ms  13.028ms  13.270ms  cudaFreeHost
                    0.83%  7.9213ms         4  1.9803ms  268.25us  2.5604ms  cudaFree
                    0.24%  2.2780ms       100  22.780us  3.9570us  1.7957ms  cudaLaunchKernel
                    0.12%  1.1424ms       400  2.8550us  2.1610us  55.301us  cudaMemcpyAsync
                    0.06%  601.74us         4  150.43us  86.952us  288.17us  cudaMalloc
                    0.02%  171.00us       114  1.5000us     100ns  74.179us  cuDeviceGetAttribute
                    0.01%  85.297us         8  10.662us  1.9210us  69.892us  cudaStreamCreate
                    0.01%  75.232us         8  9.4040us  3.0040us  51.849us  cudaStreamDestroy
                    0.00%  19.328us         1  19.328us  19.328us  19.328us  cuDeviceGetName
                    0.00%  7.1550us         1  7.1550us  7.1550us  7.1550us  cuDeviceGetPCIBusId
                    0.00%  1.5970us         3     532ns     115ns  1.2220us  cuDeviceGetCount
                    0.00%     654ns         2     327ns     127ns     527ns  cuDeviceGet
                    0.00%     570ns         1     570ns     570ns     570ns  cuDeviceTotalMem
                    0.00%     378ns         1     378ns     378ns     378ns  cuModuleGetLoadingMode
                    0.00%     284ns         1     284ns     284ns     284ns  cuDeviceGetUuid
```

```
==2847417== Profiling application: ./hw4 /home/pp25/share/hw4/testcases/t28 out.bin
==2847417== Profiling result:
            Type  Time(%)      Time     Calls       Avg       Min       Max  Name
 GPU activities:   98.63%  471.89ms         4  117.97ms  109.64ms  124.70ms  flash_attention_kernel_final(float const *, float const *, float const *, float*
)
                   1.08%  5.1798ms        12  431.65us  421.25us  447.46us  [CUDA memcpy HtoD]
                   0.28%  1.3594ms         4  339.84us  333.03us  357.57us  [CUDA memcpy DtoH]
      API calls:   71.09%  422.78ms         1  422.78ms  422.78ms  422.78ms  cudaDeviceSynchronize
                   26.44%  157.24ms         4  39.309ms  8.0456ms  132.57ms  cudaMallocHost
                    1.63%  9.6810ms         4  2.4203ms  2.2647ms  2.6531ms  cudaFreeHost
                    0.24%  1.4506ms         4  362.65us  166.87us  437.99us  cudaFree
                    0.24%  1.4350ms         4  358.75us  6.3920us  1.4133ms  cudaLaunchKernel
                    0.15%  863.58us         8  107.95us  1.9830us  842.01us  cudaStreamCreate
                    0.08%  481.91us         4  120.48us  64.330us  262.56us  cudaMalloc
                    0.06%  381.66us         1  381.66us  381.66us  381.66us  cuDeviceGetPCIBusId
                    0.03%  171.02us       114  1.5000us      97ns  70.422us  cuDeviceGetAttribute
                    0.02%  120.74us        16  7.5460us  2.6890us  53.707us  cudaMemcpyAsync
                    0.01%  68.298us         8  8.5370us  2.6020us  46.799us  cudaStreamDestroy
                    0.00%  17.745us         1  17.745us  17.745us  17.745us  cuDeviceGetName
                    0.00%  1.2890us         3     429ns     115ns     978ns  cuDeviceGetCount
                    0.00%     714ns         2     357ns     161ns     553ns  cuDeviceGet
                    0.00%     377ns         1     377ns     377ns     377ns  cuModuleGetLoadingMode
                    0.00%     347ns         1     347ns     347ns     347ns  cuDeviceTotalMem
                    0.00%     271ns         1     271ns     271ns     271ns  cuDeviceGetUuid
```

此外，由實驗結果發現，我的程式 bottleneck 是 compute time，其實算合理，因為前面一系列的優化都是在資料存取

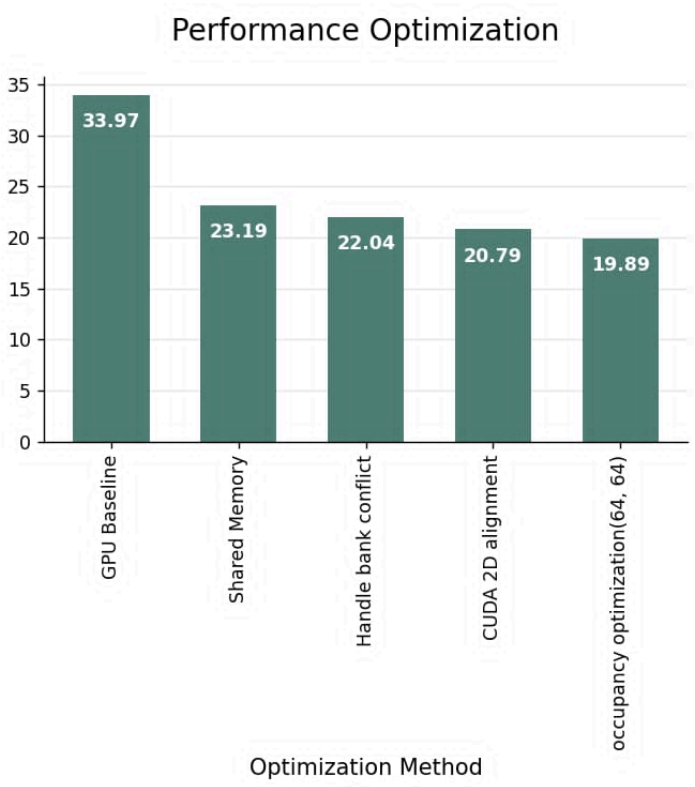
3. Experiment & Analysis

a. System Spec

我使用課堂提供的GPU與CPU

b. Optimization

此圖記錄的時間是所有測資的總時間



GPU Baseline (33.97s):
最開始的 CUDA 實作，雖然已經將計算從 CPU 轉移到 GPU，但每個執行緒在計算 Attention Score 時，都需要直接從 **Global Memory (HBM)** 讀取 K 和 V。
bottleneck：由於 Global Memory 的延遲高且頻寬有限，導致程式受限於記憶體存取 (Memory Bound)。

Shared Memory (23.19s):

```
__global__ void flash_attention_kernel_smem(
    const float* __restrict__ Q,
    const float* __restrict__ K,
    const float* __restrict__ V,
    float* __restrict__ O,
    int N, int d, float scale
) {
    // 1. 定義 Shared Memory
    // 用來暫存 Key 和 Value 的 Tile
    // 大小為 [BC][MAX_D]，對應作業 Algorithm 1 中將 K, V 搬入 SRAM 的步驟
    __shared__ float s_K[BC][MAX_D];
    __shared__ float s_V[BC][MAX_D];
```

因為SRAM 的頻寬遠高於 HBM。當資料進入 SRAM，執行緒就可以重複快速讀取，大幅降低較長的 Global Memory 存取次數。在此次優化時間大幅縮減至23.19s

Handle Bank Conflict (22.04s):

```
__shared__ float s_K[BC][MAX_D + PAD];
    __shared__ float s_V[BC][MAX_D + PAD];
```

在未加 Padding 前，不同執行緒同時存取 Shared Memory 的同一 Column 時會發生 **Bank Conflict**，導致存取被序列化。加入 Padding 後，存取位置錯開到不同的 Memory Banks，恢復了平行存取的高bandwidth。

CUDA 2D Alignment (Vectorization) (20.79s):

```
// 使用 float4 指標進行 Vectorized Load
    const float4* q_batch_vec = (const float4*)(Q + batch_offset);
    const float4* k_batch_vec = (const float4*)(K + batch_offset);
    const float4* v_batch_vec = (const float4*)(V + batch_offset);
```

使用 `float4` 資料型態進行記憶體存取，替代原本的 `float`。因為 `float4` 能讓 GPU 在單一指令週期內搬運 128-bit的資料。這減少了 Load/Store 指令的總數，並提升了 Global Memory 到 Shared Memory 的傳輸效率。

Occupancy Optimization (64, 64) (19.89s):
將 Block Size (B_r) 和 Tile Size (B_c) 從 32 增大到 **64**，並使用 `__launch_bounds__`。增加 Block Size 讓每個 SM 有更多的 Warps 可以排程，當某些 Warps 等待記憶體時，SM 可以切換到其他 Warps 繼續計算。且更大的 Tile Size ($B_c = 64$) 代表每次載入資料後，能進行更多的矩陣運算，提升了 Arithmetic Intensity，最終時間落在19.89s。

5. Experience & conclusion

a. What have you learned from this homework?

在這次作業一開始，就花了許多時間在理解attention機制與flashattention為何會比較快，之後的實作更是燒腦，可說是難度相當高的一份作業。但此次因為時間的緣故，沒有設計其他額外的實驗，稍微可惜。另外在此次作業中，我對GPU的整個memory架構又有更深的認識，深深感受到要寫好CUDA一定要弄懂他的 Memory Hierarchy這件事。